

Архитектура агентов

Как связать решения языковой модели с состоянием задачи, инструментами и проверяемыми изменениями внешней среды.

От ответа к управляемому действию

Языковая модель вычисляет продолжение входной последовательности. Агентная система использует её результат для выбора действия: прочитать документ, выполнить поиск, изменить файл или запустить проверку. После действия мир может измениться, а следующий выбор зависит от наблюдаемого результата. В *Artificial Intelligence: Foundations of Computational Agents*¹ это различие вводится через контроллер, его внутреннее состояние и среду. Команда и наступивший эффект рассматриваются отдельно.

Для цифрового агента «среда» вполне материальна: файлы имеют версии, процесс может завершиться с ошибкой, база данных меняется конкурентно, сетевой ответ иногда теряется. Качество текста модели составляет лишь часть качества системы. Требуется определить, что считается выполненной задачей, кто вправе менять результат и как продолжить работу после перерыва.

Три архитектурных решения

Первое решение — что поручить модели. Если последовательность действий известна, их можно связать программно, оставив LLM извлечение смысла или генерацию текста. Если следующее действие зависит от неожиданной находки, полезен адаптивный контроллер. *Building effective agents*² показывает такие варианты через фиксированные процессы, маршрутизацию и динамический выбор действий. Второе решение — где находится достоверное состояние. Текст «исправление завершено» в диалоге не заменяет сохранённый патч и результат проверки. Третье — как подтверждается успех. *Demystifying evals for AI agents*³ предлагает рассматривать задание, попытку, трассу и оценщик отдельно: это позволяет найти причину улучшения или отказа.

Сквозной пример

Рассмотрим агента, который исправляет поиск по корпоративным PDF: ссылка должна открывать именно страницу найденного фрагмента. Он изучает описание дефекта, находит преобразование номеров страниц, предлагает исправление и проверяет его. Пример будет развиваться через всю главу; он учебный и не является отчётом о выполненном эксперименте.

Четыре разных объекта

Модель: преобразует контекст в ответ или предложение действия.

Контроллер: решает, какое событие обрабатывать и какой шаг разрешён.

Среда исполнения: запускает инструменты и хранит результаты.

Оценщик: проверяет, выполнены ли требования.

Что означает автономия

Её надо задавать по действиям: чтение, правка, запуск, публикация. Одна шкала «умнее — автономнее» скрывает различия в полномочиях.

Подтемы главы

1. Контроллеры и планирование
2. Память и состояние задачи
3. Инструменты и контракты
4. Инфраструктура и восстановление
5. Координация нескольких агентов
6. Разработка на длинной дистанции
7. Оценка и наблюдаемость

Три вида детерминированности

Детерминированный переход контроллера означает: при одинаковом состоянии и событии выбирается одинаковая программная ветвь. Воспроизведение зафиксированной истории означает, что можно восстановить ход уже состоявшегося запуска. Повторный запуск всей системы — другой опыт: источники могли измениться, модель может выдать иной ответ, а внешний сервис — другое наблюдение. Эти свойства следует проверять отдельно.

Минимальная формальная запись помогает увидеть границы ответственности. Пусть z_t — сохранённое состояние задачи, o_t — новое наблюдение, c_t — контекст модели. Тогда проект может использовать:

$$c_t = G(z_t, o_t), \quad a_t \sim \pi_\theta(\cdot \mid c_t), \\ z_{t+1} = F(z_t, a_t, o_{t+1}).$$

Здесь G выбирает входные сведения, политика π_θ предлагает действие, а F обновляет состояние после его результата. Эта запись — обобщённая схема, а не утверждение, что все агенты устроены одинаково. Можно сделать F строгим и проверяемым, сохранив вероятностный выбор внутри π_θ .

Как изучать архитектуру

Для каждого компонента полезно спросить: какую неопределённость он снимает; что получает на вход; какой результат сохраняет; как обнаруживается его ошибка. В нашем примере поиск по коду уменьшает неопределённость о месте дефекта, тест — о поведении патча, а журнал — о том, что уже проверено. Ни один из этих компонентов не заменяет остальные.

Глава последовательно рассматривает контроллеры, память, инструменты, долговечное исполнение, координацию, длительную разработку и оценку. Поиск по корпоративному корпусу, эмбединги, ранжирование и графовые индексы имеют собственный маршрут RAG. Здесь важно, как агент вызывает этот поиск и интерпретирует его результат. Научная постановка гипотез и доказательный синтез подробно изучаются в главе Deep Research.

От проекта к сравнению

Сравнивать архитектуры следует на одной задаче и при заданном бюджете. Сначала строится понятный базовый процесс, затем изменяется один существенный компонент. Проверяется не только доля завершённых задач, но и стоимость, задержка, сохранность состояния и объяснимость отказа. Удобная схема становится обоснованным проектом, когда известны условия её применимости.

Формула по шагам

- 1. Вход:** сохранены номер задачи, версия кода и неудачный тест.
- 2. Контекст:** добавлен фрагмент функции, переводящей номер страницы.
- 3. Действие:** модель предлагает локальную правку.
- 4. Наблюдение:** тест сообщает фактический открываемый номер.
- 5. Состояние:** «проверено» записывается только вместе с результатом.

Подробное объяснение

§2.1.1–2.1.3 учебника¹:
контроллер, память и функции обновления; разобран числовой пример состояния.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Poole, D. L., Mackworth, A. K. (2023), *Artificial Intelligence: Foundations of Computational Agents*. Учебник.

Искусственный интеллект: основы вычислительных агентов

Учебник связывает устройство агента с поиском, представлением знаний и решениями при неопределённости. Контроллер получает наблюдения, поддерживает внутреннее состояние и выдаёт команды среде. Примеры позволяют отличить желаемое действие от фактически наступившего результата. Открытая авторская версия содержит математические объяснения и упражнения.

Что читать. 3-е издание, Cambridge University Press: §2.1–2.2; главы 3, 6, §9.6.2 и §12.5; глава 14 для многоагентных систем.

2. Schluntz, E., Zhang, B. (2024), *Building effective agents*. Инженерная статья.

Построение эффективных агентов

Материал вводит различие между заранее заданными маршрутами исполнения и выбором следующего действия самой моделью. На схемах показаны цепочки, маршрутизация, параллельные ветви и координатор с исполнителями. Отдельное приложение рассматривает проектирование инструментов. Это инженерный опыт разработчика моделей, а не универсальное экспериментальное сравнение архитектур.

Что читать. What are agents?; Building blocks, workflows, and agents; Appendix 2.

3. Anthropic (2026), *Demystifying evals for AI agents*. Инженерная статья.

Как устроена оценка ИИ-агентов

Разбираются задания, повторные попытки, трассы и оценщики агентных систем. Материал помогает построить проверку результата отдельно от анализа поведения и подобрать сочетание программных, модельных и человеческих оценок. Примеры помогают спроектировать регрессионный набор до улучшения промптов.

Что читать. Разделы о структуре evaluation, типах graders, метриках повторных попыток и создании evaluation suite; опубликовано 9 января.

Порядок чтения. Начать с главы 2 учебника Poole и Mackworth, затем сопоставить паттерны Anthropic с задачами собственного агента. Материал об оценке прочитать до выбора более сложной архитектуры.

1. Контроллеры и планирование

Последовательность инструкций, поиск вариантов и реакция на события задают разные способы управления агентом.

План — предположение о будущем

Агент может записать «найти функцию, исправить, проверить». Это намерение, а не доказательство возможности исполнения. Функции может не существовать, тест может зависеть от недоступного сервиса, а найденное расхождение — объясняться другой системой нумерации. Поэтому план должен содержать зависимости и условия пересмотра.

ReAct² соединяет выбор действия с наблюдениями. После чтения файла агент получает новую информацию и меняет следующий шаг. Практический смысл здесь в замкнутом контуре: действие выбирается по обновлённому контексту. Записанное рассуждение удобно для анализа, но само по себе не является надёжной записью выполненных операций или точным описанием внутренних вычислений модели.

Четыре разных графа

Граф зависимостей задач отвечает на вопрос «что должно завершиться раньше». Дерево поиска представляет альтернативные частичные решения. Диаграмма состояний задаёт допустимые режимы работы и переходы по событиям. Граф знаний хранит сущности и отношения предметной области. Их вершины имеют разный смысл; слово «граф» не делает эти конструкции взаимозаменяемыми.

В Tree of Thoughts³ контроллер хранит несколько промежуточных вариантов, оценивает их и управляет обходом. Если ширина ветвления равна b , а глубина d , полный перебор дерева быстро становится дорогим:

$$N = 1 + b + b^2 + \dots + b^d.$$

При $b = 3$ и $d = 4$ получается 121 вершина. Это учебный подсчёт узлов, не число обязательных вызовов LLM: часть операций можно объединять, а многие ветви — отсекают. Ключевой вопрос состоит в качестве оценки, определяющей отсечение.

Когда нужен автомат

Для нашего исправления достаточно состояний «изучение», «изменение», «проверка», «готово». Ошибка теста возвращает управление к изменению; отсутствие доступа переводит задачу в ожидание. Statecharts⁴ добавляет иерархию и параллельные области, если плоский автомат становится громоздким. При этом LLM может выбирать конкретную правку внутри состояния, не меняя правил перехода в «готово».

Обход дерева

1. Разветвить: три гипотезы о причине неверной страницы.

2. Проверить: дешёвым наблюдением отбросить несовместимые варианты.

3. Углубить: написать тест только для оставшейся гипотезы.

4. Вернуться: если тест опроверг её, восстановить предыдущую точку поиска.

Что прочитать

§3 Tree of Thoughts³: разбиение, генерация, оценка и поиск. Учебник: глава 3 для алгоритмической основы.

Неполная наблюдаемость

Полное состояние внешней системы обычно неизвестно. Агент видит ответ API и часть файла, но может не видеть незавершённую транзакцию или параллельную правку. Марковская модель предполагает, что выбранное состояние содержит всё необходимое для предсказания следующего перехода. Сжатое текстовое резюме не приобретает это свойство автоматически.

В §12.5 учебника Poole и Mackworth¹ решения при неопределённости вводятся через процессы принятия решений. Для POMDP состояние знания можно представить распределением $b_t(s)$. После действия a и наблюдения o оно обновляется:

$$b_{t+1}(s') = \eta P(o | s', a) \sum_s P(s' | s, a) b_t(s),$$

где η нормирует вероятности. Это строгая модель для заданной динамики и наблюдений; текстовая память LLM обычно лишь приближённо играет похожую роль.

Интуиция обновления

Допустим, причина ошибки — нумерация с нуля или устаревший индекс; обе гипотезы первоначально имеют вероятность 0,5. Проверка возвращает признак, который в учебной модели вероятен с вероятностью 0,9 при первой причине и 0,2 при второй. Если состояние не меняется от проверки, вес первой гипотезы становится $0,9 \cdot 0,5 / (0,9 \cdot 0,5 + 0,2 \cdot 0,5) \approx 0,818$. Наблюдение изменило оценку, но не дало достоверности. В реальном проекте эти вероятности потребовали бы данных.

Остановка и перепланирование

Нужны разные основания для остановки: выполнено проверяемое требование; исчерпан бюджет; отсутствует обязательный вход; работа отменена. Их нельзя сводить к одному флагу «готово». Также полезно распознавать отсутствие прогресса: повтор одинакового действия без нового наблюдения не должен бесконечно увеличивать историю. Для отладки сравнивают фиксированный процесс, ReAct-контур и поиск нескольких кандидатов. Один и тот же тест результата отделяет пользу адаптации от выгоды дополнительных попыток. Формальный контроллер ограничивает порядок работы, а качество плана по-прежнему зависит от знания среды, полезности наблюдений и способности обнаруживать ошибочные предпосылки.

Обозначения

s — возможное состояние мира.

$b_t(s)$ — вероятность состояния по накопленным наблюдениям.

$P(s' | s, a)$ — модель перехода.

$P(o | s', a)$ — модель наблюдения.

η — нормировка до суммы 1.

Вывод по шагам

Poole и Mackworth, §12.5: постановка POMDP. **§9.6.2 того же учебника:** фильтрация, предсказание и поправка по наблюдению с числовым примером.

Граница автомата

Правильная диаграмма переходов не доказывает, что выбранное моделью исправление решает задачу.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Poole, D. L., Mackworth, A. K. (2023), *Artificial Intelligence: Foundations of Computational Agents*. Учебник.

Искусственный интеллект: основы вычислительных агентов

Учебник связывает устройство агента с поиском, представлением знаний и решениями при неопределённости. Контроллер получает наблюдения, поддерживает внутреннее состояние и выдаёт команды среде. Примеры позволяют отличить желаемое действие от фактически наступившего результата. Открытая авторская версия содержит математические объяснения и упражнения.

Что читать. 3-е издание, Cambridge University Press: §2.1–2.2; главы 3, 6, §9.6.2 и §12.5; глава 14 для многоагентных систем.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Yao, S., Zhao, J., Yu, D. и др. (2022; ICLR 2023), *ReAct: Synergizing Reasoning and Acting in Language Models*. Исследовательская статья.

ReAct: совместное использование рассуждения и действий

Метод чередует текстовые промежуточные рассуждения, действия и наблюдения среды. Следующий шаг может измениться после поиска или неудачного действия. Сравнения с вариантами без рассуждения и без взаимодействия раскрывают вклад обратной связи. Приложения содержат промпты и полные траектории, пригодные для последовательного разбора.

Что читать. §2; §3.2–3.3 и §4; приложения C–D с промптами и траекториями.

3. Yao, S., Yu, D., Zhao, J. и др. (2023), *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. Исследовательская статья.

Дерево мыслей: целенаправленное решение задач с LLM

Поиск организован по промежуточным текстовым состояниям: модель предлагает продолжения и оценивает их перспективность, а алгоритм управляет обходом. Работа связывает декомпозицию шага, оценку и BFS/DFS. Эксперименты относятся к конкретным задачам, включая арифметическую игру и кроссворды; они не устанавливают эффективность дерева для любого внешнего рабочего процесса.

Что читать. §3 — четыре проектных решения; §4.1 — Game of 24; приложение A — код, промпты и траектории.

4. Harel, D. (1987), *Statecharts: A Visual Formalism for Complex Systems*. Исследовательская статья.

Диаграммы состояний: визуальный формализм сложных систем

Классическая работа расширяет диаграммы состояний иерархией, параллельными областями и компактным описанием переходов. Позволяет моделировать одновременно фазу задачи, ожидание события и состояние исполнителя. Для агентного проекта полезна как формальный язык контроллера: диаграмма делает допустимые переходы предметом обсуждения и проверки.

Что читать. Science of Computer Programming 8(3), 231–274: примеры вложенности, orthogonality и history; проследить реакцию на одно событие.

Порядок чтения. Глава 2 учебника задаёт понятия. Затем разобрать одну траекторию ReAct и один пример Tree of Thoughts. Statecharts читать для переноса правил перехода в явную модель процесса.

2. Память и состояние задачи

Полезная память сохраняет основания следующего решения: что известно, что изменилось и что ещё требуется проверить.

Что именно надо помнить

У агента по исправлению PDF-поиска есть несколько видов информации. Текущий контекст содержит доступный фрагмент кода. Рабочее состояние хранит версию репозитория, выбранную гипотезу и статус теста. Эпизодическая память фиксирует прошлые попытки; процедурная — повторно используемый способ проверки ссылок. Корпоративная база знаний хранит документы предметной области. Это логические роли данных, а не обязательное требование завести пять отдельных баз.

MemGPT¹ показывает иерархию: часть данных находится во входе LLM, остальные доступны через операции памяти. Чтобы внешняя запись повлияла на ответ, её нужно извлечь и включить в вычисление. Увеличение хранилища само по себе не улучшает выбор нужных сведений.

Сжатие как потеря различий

Представим две истории. В первой тест запущен и прошёл, во второй агент только предложил его запустить. Если обе превращены в резюме «исправление и проверка теста», после смены контекста различие исчезнет. Проблема здесь не в отсутствии ключевого слова, а в утрате статуса действия. Хорошее сжатие сохраняет различия, от которых зависит следующий шаг.

Полезно мыслить память как преобразование истории H_t в представление M_t , а извлечение — как выбор доступной части:

$$M_t = C(H_t), \quad R_t = R(M_t, q_t).$$

C может потерять нужное свидетельство; R может не найти сохранившееся. AMA-Bench² разделяет эти виды потерь экспериментально. Поэтому ошибку памяти нельзя диагностировать только по похожести найденного текста на запрос.

Граф для истории действий

Связь «тест проверяет патч», «патч основан на версии» и «результат опровергает гипотезу» помогает восстановить последовательность работы. В AMA-Agent граф дополняется поиском по словам и инструментальным обходом. Но извлечённая из текста причинная связь ещё требует проверки: название causality graph не превращает автоматическое извлечение в доказательство причинности.

Минимальная запись

Задача: исправить переход на страницу.

Основание: конкретный пример и версия файла.

Попытка: патч и его идентификатор.

Проверка: команда, вход и наблюдаемый результат.

Открытый вопрос: что не проверено после изменения.

Два разных отказа

Запись удалена при сжатии — дефект построения памяти. Запись есть, но не выбрана — дефект извлечения.

Подробный опыт

§4, таблица 3²: сопоставить исходные свидетельства, построенную память и полный поиск.

Версия и происхождение

Факт «тест проходит» относится к определённому коду, входам и окружению. После новой правки он становится историческим фактом, а не свидетельством качества текущей версии. Поэтому состояние должно ссылаться на идентификатор результата проверки. Простая пометка «актуально» слабее связи с конкретными входами.

Для противоречащих записей полезно хранить обе версии и основания замены. Если старый документ говорил об индексации с единицы, а новая реализация использует ноль, удаление старой записи без истории затруднит разбор прежних результатов. В рабочий контекст при этом достаточно передать применимую версию и краткую причину выбора.

Процедура не равна воспоминанию

Одна удачная попытка может породить слишком широкое правило: «при неверной ссылке всегда вычитать единицу». Это опасное обобщение единичного случая. Повторно используемый навык должен включать условия применимости, входы, наблюдения и проверяемый результат. Иначе память закрепит ошибку и будет воспроизводить её быстрее.

Recuris³ исследует связь навыков с рабочим состоянием и проверку их изменений. В системе различаются выполнение текущей задачи и обновление процедур между задачами; внешний механизм отбора остаётся фиксированным. Это позволяет поставить вопрос о переносе опыта без заявления, что модель самостоятельно переписывает все собственные правила.

Контролируемое обновление

Для собственного проекта можно сравнить старую процедуру M и кандидата M' на наборе разработки. Учебное правило приёмки:

$$\text{accept}(M') = \mathbb{1}\{\Delta Q_{\text{target}} > 0\} \mathbb{1}\{\Delta Q_{\text{anchor}} \geq -\varepsilon\}.$$

Первый множитель требует улучшить целевой случай, второй ограничивает потерю на ранее решаемых заданиях. Порог ε выбирают заранее с учётом неопределённости измерений. Это схема инженерной проверки, а не гарантия обобщения. Многократный подбор по одному набору постепенно переобучает процедуру на него; итоговый тест следует держать отдельно.

Качество памяти проверяется по возобновлению задачи: сохранены ли незавершённые действия, не повторён ли уже наступивший эффект, найдено ли основание решения. Длинный и красноречивый журнал может проиграть короткому структурированному состоянию, если необходимые различия в нём трудно извлечь.

Правило по шагам

- 1. Ошибка:** навык не проверяет, какая нумерация используется.
- 2. Кандидат:** добавить проверку нумерации перед правкой.
- 3. Целевой тест:** исправляет ли это исходный отказ?
- 4. Опорные задачи:** работают ли прежние случаи?
- 5. Решение:** принять версию либо оставить прежнюю.

Источник механизма

§2.3.3 Recuris³: Validation-Gated Patch Admission. Наше выражение упрощает идею для учебного примера.

Ограничение

Сохранение подтверждённых наблюдений и сохранение догадок модели должны иметь разные статусы.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Packer, C., Wooders, S., Lin, K. и др. (2023), *MemGPT: Towards LLMs as Operating Systems*. Препринт.

MemGPT: языковые модели с управляемой иерархией памяти

Система различает доступный модели контекст и внешнюю память, перемещая сведения между ними через функции. Разобраны рабочий блок, очередь сообщений, исполнитель функций и передача управления. Сравнения на диалогах и анализе документов показывают задачи, для которых такое разделение исследовалось. Статья служит конкретным входом в проектирование памяти и её интерфейсов.

Что читать. §2.1–2.4; §3.1–3.2; приложение 6.1 с инструкциями памяти.

ФРОНТИР ИССЛЕДОВАНИЙ

2. Zhao, Y., Yuan, B., Huang, J. и др. (2026), *AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications*. Препринт.

AMA-Bench: оценка памяти агентов на длинных траекториях

Бенчмарк использует истории взаимодействия агента со средой, включая машинные наблюдения. Авторы отдельно исследуют потери при построении памяти и при её извлечении. Предложенный AMA-Agent соединяет граф состояний и связей с инструментальным поиском. Работа помогает поставить опыт, различающий исчезновение нужного факта при сжатии и неспособность найти уже сохранённый факт.

Что читать. §3 — построение набора; §4 и таблица 3 — диагностика потерь; §5 — граф и извлечение.

3. Yu, Z., Wu, Y., Yin, Z. и др. (2026), *Recursive Experiential-Working Memory Evolution for Long-Horizon Agent Harnesses*. Препринт.

Рекурсивное развитие памяти опыта и рабочего состояния агентов

Recuris связывает состояние текущей задачи с выбором повторно используемых навыков. Изменения памяти допускаются после проверки по свидетельствам, а изменения процедур проходят регрессионный отбор. Базовая LLM и внешний механизм улучшения фиксированы: развивается ограниченный слой памяти и управления. Это свежий препринт, полезный для изучения контролируемого переноса опыта между задачами.

Что читать. §2.1–2.3, особенно Validation-Gated Patch Admission; §3.3–3.4 с абляциями. Версия 1 от 25 августа.

Порядок чтения. MemGPT — вход в иерархию памяти. AMA-Bench — постановка проверки потерь. Recuris — исследовательское продолжение о связи рабочего состояния, навыков и контролируемых обновлений.

3. Инструменты и контракты

Описание инструмента помогает модели выбрать действие; исполняемый контракт проверяет, допустимо ли это действие и что оно изменило.

От намерения к операции

Инструкция «проверь ссылку» слишком широка для надёжного инструмента. Следует определить идентификатор документа, его версию, номер страницы, способ нумерации и ожидаемый тип результата. Тогда модель выбирает содержательные параметры, а реализация может отвергнуть несуществующий документ или недопустимый диапазон до исполнения.

Спецификация инструментов MCP¹ задаёт обнаружение инструментов, их вызов, входные и выходные схемы. Это договорённость об обмене сообщениями. Прикладное правило «разрешено изменять только текущую рабочую копию» должно проверяться отдельно. Структурированный результат, присланный сервером, также следует отличать от текста, который LLM сгенерировала в формате JSON.

Что умеет выучить модель

В Toolformer² модель обучают использовать вызовы API через генерацию кандидатов, исполнение и отбор полезных вставок. Критерий отбора связан с предсказанием текста. Это объясняет один механизм приобретения инструментального поведения, но не задаёт автоматически права доступа, транзакционность или корректность внешнего действия.

В работающем приложении важен и сам интерфейс. SWE-agent³ исследует специальные команды просмотра, поиска и правки кода. При фиксированной модели изменение доступных действий и обратной связи меняет качество решения. Поэтому неудобный инструмент нельзя считать нейтральным проводником возможностей LLM.

Три уровня проверки

Сначала проверяется синтаксис: поля и типы. Затем предусловия: существует ли ресурс, совпадает ли версия, разрешено ли действие. После исполнения проверяются постусловия: действительно ли изменилось нужное значение и не затронут ли другой объект. Ошибка каждого уровня должна возвращать различимую причину. Иначе агент будет повторять тот же вызов или лечить неверно определённую проблему.

Пример контракта

Вход: документ, версия, индекс страницы и режим нумерации.

Предусловие: документ доступен, версия совпадает, индекс в диапазоне.

Выход: фактическая страница, статус перехода, идентификатор проверки.

Ошибка: отдельно «доступ запрещён», «версия изменилась» и «страница отсутствует».

Что прочитать

Tool, Output Schema и Error Handling¹: сопоставить пример запроса с результатом и ошибкой.

Смысловая ошибка при правильной схеме

Пусть документ содержит 200 страниц. Значение 14 проходит проверку целого числа и диапазона, но может означать индекс 14, печатную страницу 14 или четырнадцатый лист PDF. Корректная схема не устраняет неоднозначность предметного понятия. В контракте нужно зафиксировать систему отсчёта; в ответе вернуть однозначный адрес.

Полезно записывать операцию как проверяемое преобразование:

$$\{P(s, x)\} T(x) \{Q(s, s', r)\}.$$

P — предусловие для состояния s и входа x , T — инструмент, s' — новое состояние, r — ответ, Q — постусловие. Это обозначение контракта: его запись ещё не является формальным доказательством программы. Для внешнего сервиса остаются частичное исполнение и неопределённость наблюдения, которые потребуют отдельного восстановления.

Данные не получают полномочий

Текст в найденном PDF может содержать повелительное предложение. Его грамматическая форма не делает его инструкцией владельца системы. Аналогично описание инструмента, полученное от внешнего сервера, не должно самостоятельно расширять права приложения. Граница доверия должна сохраняться при чтении, сжатии и передаче сведений следующему агенту.

AgentDojo⁴ позволяет изучить эту проблему в среде с полезными задачами и внедрёнными инструкциями. Для проверки нужны два результата: сохранена ли способность выполнять разрешённую работу и удалось ли предотвратить навязанное действие. Запрет всех инструментов снизит часть рисков, но не решит задачу создания полезного агента.

Обратная связь как часть дизайна

Успешный вызов должен сообщать достаточно, чтобы выбрать следующий шаг: изменённый объект, версию, статус и ссылку на подробный результат. Огромный необработанный журнал увеличивает нагрузку на контекст; слишком короткое «ОК» скрывает различия между созданным, проверенным и опубликованным. В §2–3 SWE-agent³ эту проблему можно проследить на конкретных командах и наблюдениях.

Для учебной проверки достаточно сделать два интерфейса одной операции: неоднозначный и явно задающий версию с нумерацией. Затем сравнить типы неверных вызовов и последующих исправлений. Такой опыт показывает, исправляет ли новый контракт реальную причину ошибок, а не только меняет форму ответа.

Контракт по шагам

- 1. Проверить вход:** индекс задан явно с нуля.
- 2. Проверить состояние:** версия PDF совпадает с индексом поиска.
- 3. Исполнить:** построить адрес страницы.
- 4. Наблюдать:** открыть адрес и сверить страницу.
- 5. Вернуть результат:** адрес и свидетельство проверки.

Разбор процесса

Примеры Tools¹: путь запроса и ответа. Приложение A.1³: конкретные действия интерфейса и обратная связь.

Граница проверки

Схема описывает структуру. Предусловия, права и семантика проверяются исполняемым кодом приложения.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Model Context Protocol contributors (редакция 2025-11-25), *Tools — Model Context Protocol Specification*. Спецификация.

Инструменты: спецификация Model Context Protocol

Нормативный текст задаёт обнаружение и вызов инструментов, входную и выходную схемы, структурированные результаты и ошибки. Примеры сообщений позволяют проследить путь от tools/list до tools/call. Отдельно разобраны аннотации поведения и необходимость учитывать доверие к серверу. Изучать как контракт обмена, дополняя прикладными предусловиями и правилами доступа.

Что читать. Protocol Messages; Tool; Structured Content; Output Schema; Error Handling. Ссылка фиксирует редакцию, а не обещает последнюю версию.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Schick, T., Dwivedi-Yu, J., Dessì, R. и др. (2023), *Toolformer: Language Models Can Teach Themselves to Use Tools*. Исследовательская статья.

Toolformer: обучение языковой модели использованию инструментов

Авторы создают кандидаты вызовов API, исполняют их и отбирают по улучшению предсказания последующего текста. Полученные последовательности используются для дообучения модели. Работа раскрывает, откуда может взяться способность выбирать инструмент и его аргументы, и отделяет обучение этой способности от разработки среды исполнения.

Что читать. §2 — sampling, execution, filtering и fine-tuning; раздел с используемыми API и анализом ограничений.

3. Yang, J., Jimenez, C. E., Wettig, A. и др. (2024), *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. Исследовательская статья.

SWE-agent: интерфейсы агента с компьютером для автоматизации разработки

Работа исследует интерфейс чтения, поиска и редактирования кода как фактор качества агента при фиксированной модели. Компактные действия и информативная обратная связь сопоставляются с альтернативными интерфейсами. Абляции показывают, как параметры просмотра и защиты от неверных правок меняют поведение. Приложения раскрывают команды, конфигурацию и типичные отказы.

Что читать. §2–3; §5.1 — абляции интерфейса; приложения A.1 и B.4. Версия 3 от 11 ноября 2024.

4. DeBenedetti, E., Zhang, J., Balunović, M. и др. (2024), *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. Исследовательская статья.

AgentDojo: среда оценки атак через инструкции в данных и защит агентов

Среда соединяет полезные задачи, инструменты и контролируемые внедрения инструкций в недоверенные данные. Измеряются выполнение пользовательской задачи и достижение цели атакующего. Такая постановка обнаруживает защиту, которая выглядит безопасной лишь потому, что агент перестал выполнять работу. Кодовая среда позволяет воспроизводить одну задачу при разных атаках.

Что читать. Разделы с устройством AgentDojo, threat model, attacks и defenses; результаты сравнивать вместе с benign utility.

Порядок чтения. Начать с сообщений MCP. Toolformer объясняет обучение вызовам; SWE-agent — проектирование интерфейса; AgentDojo — проверку границы между данными и командами.

4. Инфраструктура и восстановление

Длительный процесс должен переживать завершение исполнителя, повторную доставку задания и неопределённый результат внешнего вызова.

Что находится вокруг модели

Для агента, который работает часами, требуется место хранения задач, очередь исполнения и среда запуска инструментов. Простой проект может соединить эти функции в одном процессе и одной БД; нагрузка и требования к изоляции могут потребовать разделения. Архитектуру определяют отказ и способ восстановления, а не количество сервисов.

В учебном проекте таблица задач хранит фазу, владельца выполнения и версию состояния. Журнал событий фиксирует принятые действия и результаты. Крупные PDF и логи удобнее хранить как объекты со ссылками из БД. Очередь передаёт работу исполнителям; изолированная среда ограничивает код и его ресурсы. Поисковый индекс помогает находить сведения, но не должен подменять источник истины о статусе операции.

История и повторное исполнение

Temporal Workflow Definition¹ объясняет восстановление через историю событий. Код контроллера должен согласованно воспроизводить команды; внешние вызовы выполняются отдельно. Если записанный результат обращения к LLM уже есть, восстановление не требует заново получать другой ответ на тот же исторический шаг.

Но незавершённая операция может быть повторена. Activity Definition² связывает этот факт с требованием идемпотентности. Разделение контроллера и действия не создаёт само по себе гарантии единственного внешнего эффекта. Если сервис успел изменить объект, но ответ не дошёл, новый исполнитель должен разрешить неопределённость.

Четыре обязательных различия

«Запланировано» означает намерение выполнить действие. «Отправлено» подтверждает попытку передачи. «Исполнено» требует свидетельства наступившего эффекта. «Проверено» добавляет проверку постусловия. Все четыре статуса могут относиться к одному инструментальному вызову, и их слияние скрывает место отказа.

В нашей задаче патч может быть сохранён, но проверка ещё не завершена. После сбоя новый запуск должен найти сохранённый патч и продолжить проверку. Повторная генерация исправления создаст другую версию и разрушит связь с прежним тестом. Так требования надёжности определяют устройство данных раньше выбора фреймворка.

Назначение компонентов

БД: состояние и атомарные изменения.

Журнал: последовательность событий.

Хранилище объектов: документы и большие артефакты.

Очередь: передача и ограничение нагрузки.

Исполнитель: вычисление и вызовы.

Изоляция: файловые, сетевые и ресурсные границы.

Подробный процесс

Deterministic constraints и Event History¹: связать команду, записанное событие и воспроизведение.

Идемпотентность и намерение

Повтор запроса должен означать повтор того же намерения, а не создание нового. В Making retries safe with idempotent APIs³ это выражается идентификатором клиента и операции. Сервер связывает ключ с параметрами и результатом. Тот же ключ с другими параметрами — конфликт, а не разрешение молча выполнить другое действие.

Пусть T_k — операция с устойчивым ключом k . Для её определённого эффекта требуется свойство:

$$T_k(T_k(s)) = T_k(s).$$

Одинаковый эффект не означает одинаковый момент исполнения или отсутствие повторного вычисления. Ключ должен пережить сетевой сбой; выдача нового ключа при каждой попытке уничтожает смысл дедупликации. Надёжность также зависит от срока хранения ключа и атомарности его записи вместе с результатом.

Компенсация вместо воображаемой перемотки

Если процесс изменяет несколько сервисов, единая транзакция может быть недоступна. Sagas⁴ рассматривает длительную операцию как последовательность транзакций с компенсациями. Отмена созданного ресурса способна компенсировать часть эффекта; уже доставленное уведомление нельзя сделать непрочитанным. Поэтому компенсация восстанавливает приемлемое бизнес-состояние, а не обязательно прежнее состояние мира.

Для агента это означает необходимость знать, какие действия обратимы, кто может выполнять компенсацию и какой результат она должна подтвердить. Отмена задачи должна остановить выдачу новых действий и отдельно обработать уже начатые. Сообщение «остановлено» без учёта незавершённых внешних вызовов не описывает состояние системы.

Конкурентность и нагрузка

Два исполнителя могут одновременно прочесть одну версию задачи. Условная запись «обновить только если версия равна v » позволяет обнаружить конфликт: один переход завершится, второй перечитает состояние. Временная аренда исполнения помогает освобождать зависшие задачи; токен поколения позволяет хранилищу отклонять запоздавшие записи старого владельца. Это предлагаемые проектные решения, которые нужно проверять в конкретной инфраструктуре. Предел параллельных запросов, очередь ожидания и отдельные бюджеты повторов защищают систему от самоподдерживающейся перегрузки. Восстановление проверяют намеренным завершением исполнителя в разных точках: до вызова, после эффекта и до записи результата. Цель опыта — сохранить корректное состояние без дублирования эффекта, а не просто добиться нового зелёного запуска.

Потерянный ответ

1. **Намерение:** создать один артефакт с ключом K .
2. **Эффект:** сервис создал артефакт.
3. **Сбой:** ответ не дошёл.
4. **Повтор:** отправлен тот же K и те же параметры.
5. **Разрешение:** сервис возвращает прежний результат.

Детальный разбор

Retries and semantic equivalence³: ключ, атомарность и эквивалентный ответ.

Разные гарантии

Повторная доставка задания, повтор функции и повтор внешнего эффекта — три отдельных свойства системы.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Temporal Technologies (обновляемая документация), *Temporal Workflow Definition*. Документация. Определение рабочего процесса Temporal

Документация объясняет воспроизведение истории и требования к детерминированному коду оркестрации. Внешние запросы и вызовы LLM выносятся в Activities; изменение последовательности команд требует совместимого обновления. Материал нужен для различения повторного исполнения кода контроллера и повторения внешнего эффекта.

Что читать. Deterministic constraints; Handling code changes; Versioning Workflow code and Patching; Event History по внутренней ссылке. Проверено 12 сентября 2026.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Temporal Technologies (обновляемая документация), *Activity Definition*. Документация.

Действия и повторное выполнение в Temporal

Рассматривается операция, которую исполнитель выполняет вне детерминированного кода рабочего процесса. Объясняется, почему повтор может вызвать функцию более одного раза и зачем прикладная идемпотентность. Примеры помогают выбрать границу операции и понять, какие частично выполненные действия останутся после сбоя.

Что читать. Разделы об Activity Definition и идемпотентности; пример многошагового действия. Проверено 12 сентября 2026.

3. Featonby, М. (дата на странице не установлена), *Making retries safe with idempotent APIs*. Инженерная статья.

Безопасные повторы запросов с идемпотентными API

Разбор показывает, как идентификатор намерения клиента позволяет распознать повтор операции после неопределённого сетевого результата. Обсуждаются атомарная запись ключа вместе с изменением, эквивалентный ответ, запоздавшие запросы и несовпадение параметров при повторном ключе. Примеры дают конкретную основу проектирования инструментов с внешними эффектами.

Что читать. Разделы с unique client request identifier, late arriving requests и Same client request ID, different intent.

4. Garcia-Molina, H., Salem, K. (1987), *Sagas*. Технический отчёт.

Саги: координация длительных транзакций

Длительная операция разбивается на транзакции, для которых задаются компенсирующие действия. Такой подход позволяет обсуждать восстановление, когда удерживать одну общую транзакцию слишком дорого. Для агентного процесса он даёт язык частичного завершения и компенсации уже наступивших эффектов, включая ограничения этой компенсации.

Что читать. Авторский технический отчёт CS-TR-070-87: вводный пример, формализация saga и recovery. Журнальную пагинацию не переносить на PDF отчёта.

Порядок чтения. Workflow Definition — устройство воспроизведения; Activity Definition — границы внешнего действия. Затем разобрать идемпотентный запрос у Featonby и компенсацию длительной операции в Sagas.

5. Координация нескольких агентов

Команда полезна, когда разделение работы даёт проверяемую выгоду, а результаты можно объединить без потери оснований и конфликтующих изменений.

Роль — только часть агента

Назвать участников «архитектором», «критиком» и «разработчиком» недостаточно. Нужно определить, что каждый получает, что вправе менять, какой артефакт возвращает и кто принимает результат. Если все читают один и тот же краткий ответ и повторяют его, дополнительные роли не создают независимой проверки.

AutoGen¹ представляет взаимодействие через агентов с разными возможностями и программируемый обмен сообщениями. Для архитектурного анализа особенно важны выбор следующего участника и различие между модельным ответом и исполнением. Подобная схема допускает много вариантов координации; само её существование ещё не доказывает пользу увеличения команды.

Топология следует зависимостям

В нашем примере один участник может исследовать нумерацию PDF, другой — проверить генерацию URL, третий — подготовить независимые тестовые документы. Эти ветви допускают частичную параллельность. Но запись финального патча и его проверка на общей версии должны быть согласованы. Одновременная правка одной функции требует протокола объединения, а не только обмена текстовыми отчётами.

Towards a Science of Scaling Agent Systems³ сравнивает несколько топологий и обнаруживает зависимость результата от типа задачи. В рассмотренных условиях координация может помогать параллельному сбору сведений и ухудшать выполнение последовательной задачи. Это эмпирическое основание проверять структуру работы; найденные коэффициенты не следует превращать в универсальные законы любых команд.

Цена коммуникации

Если каждый из n агентов потенциально отправляет сообщения каждому другому, существует $n(n-1)$ направленных пар. Это подсчёт возможных каналов, а не обязательный объём трафика. При росте команды важно ограничивать, какие сведения действительно нужны адресату. Полный пересказ общей истории каждому участнику увеличивает расходы и риск распространить одну ошибочную интерпретацию.

Поручение исполнителю

Цель: проверить преобразование номера страницы.

Вход: версия кода и два документа.

Граница: анализ и тесты; без изменения общей ветки.

Выход: найденный дефект, воспроизводящий пример, неопределённость.

Приёмка: другой участник воспроизводит результат.

Сравнительный источник

§3–4, версия 3³: топологии, бюджеты, результаты по типам задач.

Разбор отказа вместо оценки впечатления

В Why Do Multi-Agent LLM Systems Fail?² ошибки рассматриваются через разметку трасс: неясная спецификация, рассогласование участников, недостаточная проверка и неверное завершение. Такой подход помогает отличить ошибку локального решения от ошибки организации. Если исполнитель правильно решил не ту задачу, повышение качества его модели не исправит исходное поручение.

Координатор может принять утверждение «тест пройден», хотя проверяющий видел другую версию. Это отказ передачи основания: результат без идентификаторов входа выглядит завершённым, но неприменим к общему артефакту. Поэтому сообщение о выполнении должно ссылаться на результат, его версию и критерий проверки. Уверенность автора сообщения не заменяет эту связь.

Параллельность и ускорение

Рассмотрим собственную учебную оценку: доля f времени исходного процесса строго последовательна, остальное можно равномерно разделить между n исполнителями. Пусть $c(n)$ — накладные расходы как доля исходного времени. Тогда идеализированное ускорение:

$$S(n) = \frac{1}{f + (1 - f)/n + c(n)}.$$

При $f = 0,4$, $n = 3$ и $c(3) = 0,15$ получается $S \approx 1,33$. Три исполнителя дают здесь ускорение примерно на треть. Формула предполагает равномерную нагрузку и не учитывает изменение качества; реальные расходы надо измерять по трассам. Это не оценка из сравнительной статьи.

Независимая критика

Полезный критик получает критерий и возможность проверить артефакт. Для кода это запуск воспроизводящего теста и поиск регрессий; для вывода — проверка исходного свидетельства. Если критик заранее видит убедительный самоотчёт автора, он может наследовать его предпосылки. Разделение контекстов и порядок раскрытия информации — переменные эксперимента, а не гарантия независимости. Команду следует сравнивать с одним агентом на том же наборе задач и при сопоставимом бюджете. Измеряются не только успех и время, но и повторная работа, конфликты, ошибки передачи и качество объединения. Участник оправдан своей функцией: дополнительное имя роли не является самостоятельным улучшением архитектуры.

Подсчёт времени

1. **Исходный процесс:** 100 условных минут.
2. **Последовательная часть:** 40 минут.
3. **Параллельная часть:** $60/3 = 20$ минут.
4. **Координация:** 15 минут.
5. **Итог:** 75 минут; $100/75 \approx 1,33$.

Где разбирать процесс

§4.1–4.3³: измерение расходов и анализ трасс. Формула слева — упрощённая учебная декомпозиция.

Граница вывода

Более быстрое выполнение и более правильный результат — разные критерии.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Wu, Q., Bansal, G., Zhang, J. и др. (2023), *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. Исследовательская статья.

AutoGen: приложения LLM на основе взаимодействия нескольких агентов

Система задаёт агентов через возможности и способы обмена сообщениями, позволяя программировать порядок взаимодействия. Примеры включают выполнение кода и выбор следующего участника разговора. Архитектура отделяет роль, инструментальные возможности и логику коммуникации. Читать как исследовательский прототип и материал по проектированию, а не как инструкцию к актуальной версии пакета.

Что читать. §2 — conversable agents и conversation programming; §3, особенно A5–A6; приложение с конфигурациями.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Cemri, M., Pan, M. Z., Yang, S. и др. (2025), *Why Do Multi-Agent LLM Systems Fail?*. Препринт.

Почему многоагентные LLM-системы терпят неудачу

Авторы размечают отказы многоагентных систем и выделяют проблемы задания ролей, взаимодействия, проверки и завершения. Таксономия переводит общее впечатление от неудачного диалога в конкретные точки диагностики. Для изучения полезно проследить переход от трассы к категории ошибки и сравнить предложенные вмешательства. Выводы относятся к исследованным системам и заданиям.

Что читать. Версия 1: методология разметки, таксономия и interventions; примеры ошибок в приложениях.

ФРОНТИР ИССЛЕДОВАНИЙ

3. Kim, Y., Gu, K., Park, C. и др. (2025; редакция 2026), *Towards a Science of Scaling Agent Systems*. Препринт.

К количественному исследованию масштабирования агентных систем

Сопоставляются один агент и несколько топологий координации при контроле инструментов и вычислительных бюджетов. Результат зависит от структуры задачи: возможности параллельной работы и последовательные зависимости меняют пользу координации. Метрики накладных расходов и распространения ошибок помогают анализировать эту зависимость. Формулы и численные оценки следует читать вместе с областью эксперимента.

Что читать. Версия 3 от 8 апреля 2026: §3 и §4.1–4.3; ограничения. Не смешивать таблицы разных редакций.

Порядок чтения. AutoGen читать как язык организации взаимодействий. Затем использовать таксономию отказов для разбора трассы; количественные сравнения архитектур изучать по одной зафиксированной редакции работы Kim и соавторов.

6. Разработка на длинной дистанции

Продолжение через несколько сессий требует восстанавливаемого проекта: цель, решения, код и результаты проверок должны оставаться согласованными.

Документы обслуживают разные решения

Для длительной разработки нужно различать цель продукта и текущее состояние реализации. PRD описывает, для кого создаётся функция и какой результат нужен. Спецификация фиксирует поведение, границы и критерии приёмки. План раскладывает работу на зависимости; журнал прогресса сообщает, что фактически произошло. Эти документы могут быть короткими и объединяться в небольшом проекте, но их логические роли сохраняются.

Documenting Architecture Decisions¹ предлагает записывать отдельное значимое решение вместе с контекстом и последствиями. В нашем примере ADR объяснит, почему внутренний индекс страниц хранится с нуля, а в пользовательской ссылке применяется другое представление. Такая запись предотвращает будущую «коррекцию», которая случайно вернёт старый дефект.

Передача между сессиями

Effective harnesses for long-running agents² описывает начальную настройку проекта, перечень функций, журнал и восстановление контекста через git и проверку среды. Польза процедуры в том, что новый сеанс читает проверяемые артефакты. Он не вынужден достраивать историю по последнему уверенному сообщению предыдущего сеанса.

Для нашей задачи запись передачи содержит требование к переходу на страницу, ветку и версию кода, незавершённый патч, результат последнего теста и следующий нерешённый вопрос. Если проверка ещё работает, это отдельное незавершённое действие с идентификатором. Новый сеанс сначала выясняет его исход; только затем решает, нужно ли повторение.

Малая завершённая единица

Единица работы должна быть достаточно небольшой, чтобы изменить, проверить и оставить её в понятном состоянии. Это может быть исправление одного преобразования вместе с воспроизводящим тестом. Но фиксированный размер шага не универсален: слишком мелкое деление увеличивает расходы на восстановление контекста, слишком крупное — риск потерять связность изменений.

Для сравнения нужен простой маршрут. Agentless³ организует локализацию, исправление и проверку без свободного выбора всех следующих действий моделью. Он показывает, почему в исследовании агента следует отдельно оценивать пользу адаптивности, поиска кандидатов и качества тестов.

Минимальный комплект

Требование: наблюдаемое поведение и границы.

ADR: причины важного выбора.

План: зависимости и критерий завершения.

Инструкции репозитория:

запуск, соглашения, допустимая область изменений.

Журнал: версия, изменения, проверки, открытые вопросы.

Заполненные примеры

Статья Nygard¹ сама оформлена как ADR. Feature list² содержит пример проверяемого требования.

Качество на границе сессии

Перед завершением рабочего сеанса полезно связать изменение с наблюдаемым результатом. «Добавлен тест» сообщает о файле; «тест выполнен на версии X и прошёл» сообщает о проверке. «Все тесты проходят» без команды и окружения трудно использовать при расследовании. Отчёт должен сохранять именно те различия, которые потребуются следующему действию.

Инструкции репозитория задают стабильные правила, а журнал содержит изменяющееся состояние. Смещение их в одном бесконечно растущем файле затрудняет извлечение актуального. Секреты и временные токены не нужны для содержательной передачи проекта; достаточно описания способа получения разрешённого доступа.

От исправления до эксплуатации

Полный цикл включает исходное требование, локальное воспроизведение, изменение, проверки, рецензирование, интеграцию, выпуск и наблюдение в эксплуатации. Для научного кода к нему добавляются версии данных и окружения. Проверка до слияния должна относиться к той версии, которая действительно интегрируется. Изменение зависимости или параллельный коммит могут сделать прежний результат неприменимым.

В CI полезно отделять быстрые проверки от дорогих сценариев и хранить результаты с артефактами. После выпуска проверяется поведение, ради которого делалось изменение. В примере с PDF это фактический переход к нужному фрагменту на репрезентативных документах. Обратимое развёртывание и наблюдение отказов — самостоятельные части проекта, а не автоматическое следствие успешной генерации кода.

Роль оценщика меняется

Harness design for long-running application development⁴ описывает последовательное упрощение среды управления при изменении возможностей модели. Отдельно обсуждаются поверхностная проверка и склонность оценщика оправдывать найденные дефекты. Практический вопрос: какой именно отказ устраняет каждый компонент и сохраняется ли эта потребность после обновления модели?

Оценщику нужен критерий, доступ к артефакту и возможность предъявить проверяемое возражение. Фраза «выглядит хорошо» не заменяет воспроизведение сценария. При этом дополнительная роль тоже имеет стоимость и собственные ошибки. Полезность рецензента измеряется обнаруженными проблемами, ошибочными блокировками и влиянием на итоговую работу.

Предложенный комплект документов — проектная рекомендация, а не обязательный стандарт всех систем. Его можно сокращать, пока сохраняется способность нового исполнителя понять цель, восстановить состояние и проверить следующий шаг без домысливания утраченной истории.

Одна передача работы

Сделано: исправлено преобразование индекса страницы.

Проверено: два файла, конкретная версия кода.

Не проверено: сканы с печатной нумерацией, отличной от листов PDF.

Дальше: добавить такой документ и проверить адрес.

Как читать кейс

Iterating on the harness⁴:
проследить удаление одного компонента и проверку последствий.

Проверяемый спор

Критик указывает дефект, условие его проявления и способ проверки исправления. Это полезнее общей строгости тона.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Nygard, M. (2011), *Documenting Architecture Decisions*. Методический материал.

Документирование архитектурных решений

Предложен короткий документ для одного значимого архитектурного решения: контекст, выбор, статус и последствия. Сохранение причин позволяет следующему участнику понять условия, при которых решение было разумным. Сама статья оформлена как пример ADR. Для длительной агентной разработки это образец памяти о проектных решениях, отличной от перечня выполненных задач.

Что читать. Полный текст, особенно Context, Decision, Status, Consequences; опубликовано 15 ноября.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Young, J. (2025), *Effective harnesses for long-running agents*. Инженерная статья.

Организация длительной работы агентов через несколько контекстов

Разбор веб-разработки связывает начальную настройку проекта, список функций, журнал прогресса и проверку при возобновлении сессии. Показаны преждевременное завершение и потеря состояния незаконченной работы. Материал содержит пример feature list и ссылку на реализацию. Его процедуры следует адаптировать к типу проекта и проверять по реальным результатам.

Что читать. Environment management; Incremental progress; Testing; Getting up to speed. Опубликовано 26 ноября.

3. Xia, C. S., Deng, Y., Dunn, S., Zhang, L. (2024), *Agentless: Demystifying LLM-based Software Engineering Agents*. Исследовательская статья.

Agentless: анализ необходимости сложных агентов для исправления кода

Работа строит простой процесс локализации проблемы, генерации исправлений и проверки патчей. Она даёт важный базовый вариант для сравнения со свободным выбором действий агентом. Анализ задач также обнаруживает ограничения описаний и состава бенчмарка. Полезно разделять качество модели, эффект поиска кандидатов и свойства самой процедуры проверки.

Что читать. Описание localization, repair и patch validation; абляции и анализ SWE-bench Lite. Версия 2 от 29 октября.

ФРОНТИР ИССЛЕДОВАНИЙ

4. Rajasekaran, P. (2026), *Harness design for long-running application development*. Инженерная статья.

Проектирование среды управления длительной разработкой приложений

Автор сравнивает организацию планирования, генерации и оценки в длительной разработке. Далее последовательно удаляет компоненты, проверяя, какие нужны при изменении возможностей модели. Обсуждаются поверхностная проверка и чрезмерная снисходительность оценщика. Это подробный инженерный кейс с ограниченным набором приложений, а не доказательство превосходства одной схемы во всех проектах.

Что читать. Why naive implementations fall short; Iterating on the harness; Removing the sprint construct; результаты и расходы. Опубликовано 24 марта.

Порядок чтения. Nygard — память о решениях; Young — конкретные файлы и возобновление работы. Agentless задаёт простой сравнительный процесс. Кейс 2026 года читать как проверку необходимости компонентов при смене модели.

7. Оценка и наблюдаемость

Надёжность агента измеряется по повторяемым результатам и соблюдению условий задачи, а причины ошибок устанавливаются по траекториям.

Что считать успехом

В задаче с PDF недостаточно получить хорошо написанный ответ. Требуется открыть правильную страницу, сохранить корректную связь с документом и не испортить другие случаи. Проверка должна наблюдать это поведение непосредственно. Demystifying evals for AI agents¹ различает программные, модельные и человеческие оценщики: каждый подходит к определённым свойствам результата и имеет собственные ограничения.

t-bench² проверяет взаимодействие с пользователем и инструментами через итоговое состояние БД и необходимые сведения в ответе. Авторы отмечают, что даже правильный итоговый результат может не зафиксировать нарушение промежуточной процедуры. Поэтому конечное состояние и допустимость траектории — разные измерения качества.

Один успех и надёжное повторение

Пусть одна попытка решает задачу с вероятностью p , а попытки независимы и одинаково распределены. Тогда:

$$\text{pass}@k = 1 - (1 - p)^k, \quad \text{pass}^k = p^k.$$

Первая величина спрашивает, найден ли хотя бы один успех; вторая — успешны ли все попытки. При $p = 0,8$ и $k = 3$ получаются 0,992 и 0,512. Это учебные числа. Высокая вероятность найти удачный вариант после нескольких попыток не означает, что каждое обращение пользователя надёжно.

В реальном наборе заданий вероятности различаются. Поэтому нельзя подставлять средний успех во вторую формулу и считать результат оценкой всего набора. Нужны повторения по заданиям и явно описанная процедура агрегирования. Разделение данных на разработку и итоговую проверку предотвращает подбор правил на тех же случаях, по которым затем заявляется качество.

Длительность задачи

Measuring AI Ability to Complete Long Tasks³ определяет горизонт через человеческое время выполнения и вероятность успеха модели. Это способ нормировать сложность по трудозатратам человека. Он не равен времени непрерывного сеанса агента и не обещает автономной работы такой длительности на произвольном проекте.

Две вероятности

Хотя бы один успех: все три попытки неудачны с вероятностью $0,2^3$; вычитаем её из 1.

Все успешны: перемножаем $0,8 \cdot 0,8 \cdot 0,8$.

Подробный разбор

Метрика повторов в §3²: определения и оценивание повторов.

Нужна оговорка

Независимость может нарушаться общим кэшем, состоянием, источником ошибки или переносом памяти между попытками.

Трасса должна объяснять отказ

Минимальная трасса связывает задачу, попытку, версию модели, конфигурацию, действие, вход инструмента, наблюдение и результат проверки. Идентификаторы позволяют собрать историю, когда исполнителей несколько. При этом не требуется выдавать каждое промежуточное рассуждение за точное описание внутреннего механизма LLM. Для инженерного анализа особенно ценны наблюдаемые решения и эффекты.

В нашем примере неправильная страница может возникнуть в извлечении фрагмента, сохранении метаданных, построении ссылки или интерфейсе просмотра. Общий балл «ответ неверен» обнаружит проблему, но не локализует её. Раздельные проверки позволяют понять, какой компонент менять и какие прежние результаты нужно перепроверить.

Проверка длительной траектории

AgentLAB⁴ исследует атаки через последовательность взаимодействий, включая постепенное смещение цели и отравление памяти. Одна безопасно выглядящая операция может стать звеном нежелательной цепочки. Для оценки важно сохранять связь действия с исходной задачей и учитывать изменения памяти между шагами.

Практический набор испытаний включает обычное выполнение, некорректный инструментальный ответ, устаревшую версию документа, потерю результата вызова и конфликт записи. Отдельная группа проверяет недоверенные инструкции в данных. Это предложенная программа испытаний для нашего примера; она не является новым опубликованным бенчмарком.

Что должно сопровождать сравнение

Нужно зафиксировать набор заданий, начальное состояние, число повторов, модель, инструменты, бюджет и правила остановки. Стоимость успешного результата можно считать как суммарную стоимость всех попыток, делённую на число успешно завершённых задач, если именно такой режим соответствует применению. При нуле успехов эта величина не определена; её нельзя заменять нулём.

Среднее время скрывает длинные задержки, поэтому рядом полезны квантили и причины ожидания. Изменение промпта, схемы памяти, модели или версии инструмента создаёт новую конфигурацию. Регрессионный набор показывает, какие прежние задачи она перестала решать; отдельный набор развития позволяет искать новые возможности.

Глава задаёт архитектурный каркас для дальнейшего углубления. Планирование, память, распределённое исполнение и оценка имеют собственную обширную литературу. Переход к следующему уровню состоит в воспроизведении отдельных сравнений и расширении местных библиографий, сохраняя различие между установленным результатом, инженерным решением и исследовательской гипотезой.

Полезные показатели

Доля выполненных задач и устойчивость по повторам.
Нарушения условий задачи и лишние эффекты.
Стоимость, медиана задержки и длинный хвост.
Число повторов, конфликтов и незавершённых действий.
Качество после восстановления и обновления модели.

Источник длительных проверок

§3–4 AgentLAB⁴: модель угроз и устройство многошаговых испытаний.

Статус примеров

Числа в этой главе иллюстрируют логику расчёта. Системы и бенчмарки здесь не запускались.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Anthropic (2026), *Demystifying evals for AI agents*. Инженерная статья.

Как устроена оценка ИИ-агентов

Разбираются задания, повторные попытки, трассы и оценщики агентных систем. Материал помогает построить проверку результата отдельно от анализа поведения и подобрать сочетание программных, модельных и человеческих оценок. Примеры помогают спроектировать регрессионный набор до улучшения промптов.

Что читать. Разделы о структуре evaluation, типах graders, метриках повторных попыток и создании evaluation suite; опубликовано 9 января.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

2. Yao, S., Shinn, N., Razavi, P., Narasimhan, K. (2024), *t-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. Исследовательская статья.

t-bench: оценка взаимодействия инструментов, агента и пользователя

Бенчмарк объединяет симулируемого пользователя, доменные правила, инструменты и изменяемую базу данных. Качество проверяется по конечному состоянию и необходимой информации в ответе. Показатель надёжности требует успеха в каждой попытке, раскрывая непостоянство поведения. Авторы отдельно отмечают, что итоговая награда может не обнаруживать некоторые нарушения процедуры.

Что читать. §3: Task instances, Reward и метрика надёжности повторов; экспериментальная методика. Это исходный t-bench, не последующие поколения.

3. Kwa, T., West, B., Becker, J. и др. (2025), *Measuring AI Ability to Complete Long Tasks*. Исследовательская статья.

Измерение способности ИИ выполнять длительные задачи

Работа связывает вероятность выполнения задания с временем, которое на него тратят люди с соответствующей квалификацией. Горизонт при заданной вероятности успеха позволяет сопоставлять задания разной длительности. Ограничения раскрывают границы переноса: человеческое время не равно длительности непрерывного запуска агента.

Что читать. Task-completion time horizon; человеческие измерения; статистическая модель; ограничения.

ФРОНТИР ИССЛЕДОВАНИЙ

4. Jiang, T., Wang, Y., Liang, J., Wang, T. (2026), *AgentLAB: Benchmarking LLM Agents against Long-Horizon Attacks*. Препринт.

AgentLAB: оценка агентов при многошаговых атаках

Работа исследует атаки, развивающиеся через последовательность взаимодействий: смещение цели, внедрение задач и отравление памяти. Модель угроз различает управляющего запросами пользователя и атакующего через среду. Бенчмарк позволяет проверить, как опасная последовательность складывается из отдельных внешне допустимых действий. Разбор полезен для проектирования оценки всей траектории.

Что читать. §3 — threat model; §4.1–4.2 — компоненты и типы атак; приложения с примерами.

Порядок чтения. Сначала выбрать результат и оценщик по обзору Anthropic. Затем разобрать метрику t-bench, понятие горизонта METR и многошаговую модель угроз AgentLAB. Не объединять их баллы в одну универсальную шкалу.